

SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE NAME: INTERNET PROGRAMMING

COURSE CODE: CSA43

COURSE OUTCOME COVERED

CO 5: Construct interoperable web services by leveraging JAX-RPC, WSDL, SOAP, and XML Schema for seamless data exchange across distributed systems. (L4, L5)

Assessment Tool: Alternative Solution Design

Weightage: 20

Total Marks: 25

Code of Conduct

/ Adhil Bijukumar (192411133) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: Adhil Bijukumar

Question 1: REST vs SOAP for a Mobile and Web Application

A company needs to develop a web service for a mobile and web application. This requires analyzing both REST and SOAP approaches, designing an appropriate solution by selecting one, and justifying the choice based on performance, scalability, and security requirements.

1.1 Analysis of REST

REST (Representational State Transfer) is an architectural style that uses standard HTTP methods (GET, POST, PUT, DELETE) to operate on resources identified by URIs. It is stateless, meaning each request from a client contains all the information needed for the server to process it, and it typically exchanges data in lightweight formats such as JSON.

- **Performance:** REST is lightweight because JSON payloads are smaller than XML/SOAP envelopes, resulting in faster parsing and lower bandwidth usage, which is critical for mobile networks.
- **Scalability:** Statelessness allows REST services to be scaled horizontally with ease, since any server instance can handle any request without session affinity.
- **Security:** REST relies on transport-level security (HTTPS/TLS) and token-based mechanisms such as OAuth2 and JWT, which are simple to implement but offer no built-in message-level security.
- **Flexibility:** REST supports multiple data formats (JSON, XML) and integrates naturally with mobile SDKs, browsers, and modern front-end frameworks.

1.2 Analysis of SOAP

SOAP (Simple Object Access Protocol) is a protocol that uses XML-based messaging and relies on a strict contract defined by WSDL (Web Services Description Language). It supports advanced features such as WS-Security, WS-AtomicTransaction, and built-in error handling through SOAP Faults.

- **Performance:** SOAP messages carry heavier XML envelopes with additional headers, resulting in larger payloads and slower processing compared to REST.
- **Scalability:** SOAP can be stateful, which complicates horizontal scaling, though it is still usable in distributed enterprise systems with proper design.
- **Security:** SOAP has built-in, message-level security (WS-Security) supporting encryption, digital signatures, and identity federation, making it more robust for highly sensitive transactions.

- **Reliability:** Features like WS-ReliableMessaging make SOAP suitable for scenarios that demand guaranteed delivery and formal contracts (e.g., legacy enterprise or financial integrations).

1.3 Comparison Summary

Criteria	REST	SOAP
Data Format	JSON/XML (lightweight)	XML only (verbose)
Performance	Faster, low bandwidth	Slower, higher overhead
Scalability	High (stateless)	Moderate (can be stateful)
Security	Transport-level (HTTPS, OAuth2)	Message-level (WS-Security)
Best Suited For	Mobile & web apps, public APIs	Enterprise, banking, legacy systems

1.4 Proposed Solution and Justification

For a company building a service that must support both mobile and web applications, REST is the recommended approach. Mobile devices operate under constrained bandwidth and battery conditions, so REST's lightweight JSON payloads and stateless architecture directly reduce latency and data usage. REST also scales more easily under simultaneous access from many mobile and web clients because no server-side session state needs to be maintained. For security, REST APIs can be secured adequately using HTTPS/TLS in transit combined with OAuth2/JWT-based authentication and authorization, which is an industry-standard approach for consumer-facing mobile and web applications. SOAP would only be preferable if the company required strict formal contracts, ACID-compliant distributed transactions, or regulatory-grade message-level security (e.g., in banking or insurance back-end integrations) — in which case a hybrid approach (REST for client-facing APIs, SOAP for specific back-end enterprise integrations) can be adopted as an alternative design.

Question 2: Scalable Service Architecture for High Concurrent Usage

An online platform expects a large number of users accessing its services simultaneously. This requires designing a scalable service architecture that shows how the client, server, API, and database interact, along with an alternative design to further improve scalability.

2.1 Baseline Architecture (Client–Server–API–Database)

- **Client Layer:** Web browsers and mobile apps send HTTP/HTTPS requests to the platform. Clients communicate only with the API layer, never directly with the database.
- **API Layer (Application Layer):** A set of RESTful API endpoints (or an API Gateway) receives client requests, performs authentication/authorization, applies business logic, and forwards data requests to the server/application tier.
- **Server Layer:** Application servers process business logic, handle sessions where needed, and communicate with the database layer to fetch or persist data.
- **Database Layer:** Stores persistent data (user profiles, transactions, content). The server issues queries to the database and returns processed results back through the API to the client.

In this baseline flow: Client → API Gateway → Application Server → Database → Application Server → API Gateway → Client. This linear design works for moderate loads but becomes a bottleneck when user concurrency grows, since a single application server and single database instance have finite processing capacity.

2.2 Alternative Design: Scalable Distributed Architecture

To handle large-scale concurrent access, the baseline design should be enhanced using horizontal scaling, caching, and asynchronous processing:

- **Load Balancer:** Placed in front of multiple application server instances to distribute incoming client requests evenly, preventing any single server from being overwhelmed.
- **Horizontal Scaling with Microservices:** Instead of one monolithic server, the application is split into independently scalable microservices (e.g., user service, order service, notification service), each scaled based on its own demand.
- **Caching Layer (e.g., Redis/Memcached):** Frequently accessed data is cached in-memory to reduce repeated database hits, significantly lowering response time for read-heavy operations.
- **Database Scaling:** Use read replicas for read-heavy workloads and database sharding/partitioning to distribute write load across multiple database nodes. A Content Delivery Network (CDN) can also serve static assets closer to users.

- Message Queues (e.g., Kafka/RabbitMQ): Long-running or non-critical tasks (emails, notifications, analytics) are offloaded to asynchronous queues so the API layer stays responsive to real-time user requests.
- API Gateway with Rate Limiting: Manages authentication, throttling, and request routing centrally, protecting back-end services from overload and abuse.

2.3 Justification

This alternative design improves scalability by removing single points of failure and bottlenecks at every layer: the load balancer and microservices allow the compute layer to scale horizontally with demand, the caching layer reduces database load for repeated reads, database replication/sharding increases the platform's write and read throughput, and message queues decouple slow background tasks from the fast request/response cycle. Together, these changes allow the platform to serve a much larger number of simultaneous users while maintaining low latency and high availability compared to the single-server, single-database baseline design.

Question 3: Secure Communication Protocol for a Banking Application

A banking application requires secure communication between client and server. This requires analyzing available communication protocols and selecting the most suitable one, justified on the basis of security, reliability, and performance.

3.1 Available Protocol Options

- HTTP: Plain text protocol; not suitable for banking as data (credentials, transaction details) would be transmitted unencrypted and vulnerable to interception.
- HTTPS (HTTP over TLS/SSL): Encrypts data in transit using TLS, provides server authentication via digital certificates, and protects against eavesdropping and man-in-the-middle attacks.
- SOAP with WS-Security: Provides message-level security including XML encryption and digital signatures, useful for enterprise-to-enterprise banking integrations (e.g., core banking to payment networks).
- WebSocket (secured as WSS): Enables real-time, full-duplex communication, useful for live features like fraud alerts or live balance updates, but must be layered over TLS for security.

3.2 Selected Protocol and Justification

For client-to-server communication in a banking application (mobile app or web portal to the bank's servers), HTTPS (HTTP over TLS 1.2/1.3) is the most suitable protocol, optionally combined with SOAP/WS-Security for specific back-end, system-to-system financial transactions that require message-level, non-repudiable security.

- Security: TLS encrypts all data in transit end-to-end, uses certificate-based server authentication to prevent impersonation/phishing, and can be combined with mutual TLS (mTLS) or OAuth2/JWT-based authentication and multi-factor authentication for strong client verification.
- Reliability: HTTPS runs over TCP, which guarantees ordered, reliable delivery of data; combined with retry logic and idempotent API design, this ensures transaction requests are not lost or duplicated.
- Performance: Modern TLS 1.3 significantly reduces handshake overhead compared to older TLS versions, keeping latency low while maintaining strong encryption, which is important for a responsive banking user experience.

As an alternative/complementary design, sensitive back-end integrations (e.g., core banking system to a payment gateway or interbank settlement network) can use SOAP with WS-Security over HTTPS, since it provides message-level encryption and digital signatures that persist even if

the message passes through intermediate systems, offering stronger non-repudiation guarantees than transport-level security alone.

Question 4: API Integration Strategy for Third-Party Services

An application needs to integrate with third-party services such as payment gateways or maps. This requires designing an API integration strategy covering request handling, authentication, and response processing, along with an alternative approach to improve efficiency or reliability.

4.1 Request Handling

- Use a dedicated integration/service layer (an API client module or microservice) that isolates all third-party calls from the core application logic, so changes to the third-party API only affect one layer.
- Validate and sanitize outgoing request data (e.g., payment amount, currency, map coordinates) before sending it to the third-party service to prevent malformed requests.
- Apply timeouts and retry policies (with exponential backoff) for transient network failures when calling external APIs such as a payment gateway.

4.2 Authentication

- Use the authentication scheme mandated by the third-party provider — typically API keys, OAuth2 access tokens, or signed requests (HMAC) for payment gateways.
- Store API keys/secrets securely (e.g., in a secrets manager or environment-level vault), never in client-side code or version control.
- Refresh OAuth2 tokens automatically before expiry to avoid failed calls during peak transaction periods.

4.3 Response Processing

- Parse and validate the third-party response structure before using it (e.g., confirm a payment gateway's transaction status field before marking an order as paid).
- Log response metadata (status codes, transaction IDs) for auditing and troubleshooting, especially for financial transactions.
- Map third-party error codes to standardized internal error responses so the rest of the application does not need to understand every external provider's error format.

4.4 Alternative Approach to Improve Efficiency/Reliability

Instead of calling each third-party service directly and synchronously from every part of the application, an API Gateway / Integration Hub pattern can be adopted: all third-party calls are routed through a central gateway that handles authentication, caching of non-sensitive responses (e.g., map tile data), circuit breaking (to stop calling a provider that is failing repeatedly), and rate limiting. For non-time-critical operations (e.g., sending payment confirmation receipts), requests can be queued asynchronously using a message broker, so temporary third-party outages do not

block the user-facing request. This combination of a central gateway, circuit breaker pattern, and asynchronous queuing improves both efficiency (through caching and reduced duplicate calls) and reliability (through isolation of failures and controlled retries).

Question 5: Error Handling and Fault Tolerance Mechanism

A web service frequently encounters errors during client requests. This requires designing an error handling and fault tolerance mechanism, along with alternative strategies to improve system reliability and ensure a smooth user experience.

5.1 Error Handling Design

- **Standardized Error Responses:** Define a consistent error response format (status code, error code, human-readable message) across all API endpoints, using appropriate HTTP status codes (4xx for client errors, 5xx for server errors) or SOAP Fault elements for SOAP-based services.
- **Input Validation:** Validate all incoming requests at the API boundary and return clear, specific error messages for invalid data before it reaches business logic.
- **Centralized Exception Handling:** Use a centralized exception-handling layer/middleware so that unhandled exceptions are caught, logged, and converted into safe, standardized error responses rather than exposing internal stack traces to clients.
- **Logging and Monitoring:** Log all errors with contextual information (timestamp, request ID, user ID) and integrate with a monitoring/alerting system so failures are detected quickly.

5.2 Fault Tolerance Mechanisms

- **Retry with Backoff:** Automatically retry transient failures (e.g., temporary network glitches) a limited number of times with exponential backoff to avoid overwhelming a struggling service.
- **Circuit Breaker Pattern:** Temporarily stop sending requests to a downstream service that is repeatedly failing, allowing it time to recover, and periodically test if it has recovered before resuming normal traffic.
- **Failover and Redundancy:** Deploy redundant service instances and databases (with replication) across multiple servers or availability zones so that if one instance fails, traffic is automatically redirected to a healthy instance.
- **Graceful Degradation:** When a non-critical dependency fails (e.g., a recommendation service), the application continues to function with reduced functionality rather than failing the entire request.

5.3 Alternative Strategy to Improve Reliability and User Experience

As an alternative/complementary strategy, the system can adopt asynchronous, event-driven error recovery: instead of forcing the client to wait for a failing operation to complete synchronously, the request is accepted immediately (with an acknowledgement and a tracking ID), queued, and processed in the background with automatic retries; the client is then notified of the final outcome

(e.g., via push notification, webhook, or polling a status endpoint). This is combined with a fallback/cache strategy — serving the last known good response from a cache when a live service is temporarily unavailable — so that users experience degraded but functional service rather than a hard failure. Together, standardized error handling, circuit breakers, redundancy, and asynchronous recovery significantly improve fault tolerance and provide a smoother, more resilient user experience.